Clean Beauty Recommender Project 3: Dimensionality Reduction & Product Embeddings (A Continuation of Projects 1 & 2)

Sheriann McLarty

2025-07-07

Contents

Introduction

This project is the **third installment** of a broader recommender system series centered on clean beauty and algorithmic fairness. Building on **Project 1 (bias-aware collaborative filtering)** and **Project 2 (rule-based filtering with safety flags)**, we now shift into the **math-heavy space of dimensionality reduction** and **product similarity visualization**.

Our ultimate goal is to create *product embeddings*—numerical representations of products that capture their underlying characteristics and relationships in a multi-dimensional space. These embeddings allow us to quantify similarity between products, even when they haven't been rated by the same users.

To generate these embeddings, we first apply **Truncated SVD** to uncover latent features from our user rating data. Then, to visualize these high-dimensional embeddings in an interpretable way, we use **t-SNE** to project them into a 2D space.

Load & Prepare the Dataset

```
df <- read_csv("filtered_skintone_reviews_r.csv")
```

```
## Warning: One or more parsing issues, call `problems()` on your data frame for details,
## e.g.:
##   dat <- vroom(...)
##   problems(dat)
```

```
## Rows: 42715 Columns: 45
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr  (20): review_text, review_title, skin_tone, eye_color, skin_type, hair_...
## dbl  (24): Unnamed: 0, author_id, rating_x, is_recommended, helpfulness, tot...
## date  (1): submission_time
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
ratings_df <- df %>%
  select(user = author_id, product = product_name_x, rating = rating_x) %>%
  filter(!is.na(user), !is.na(product), !is.na(rating)) %>%
  mutate(user = as.factor(user), product = as.factor(product))
```

Create a Sparse Ratings Matrix

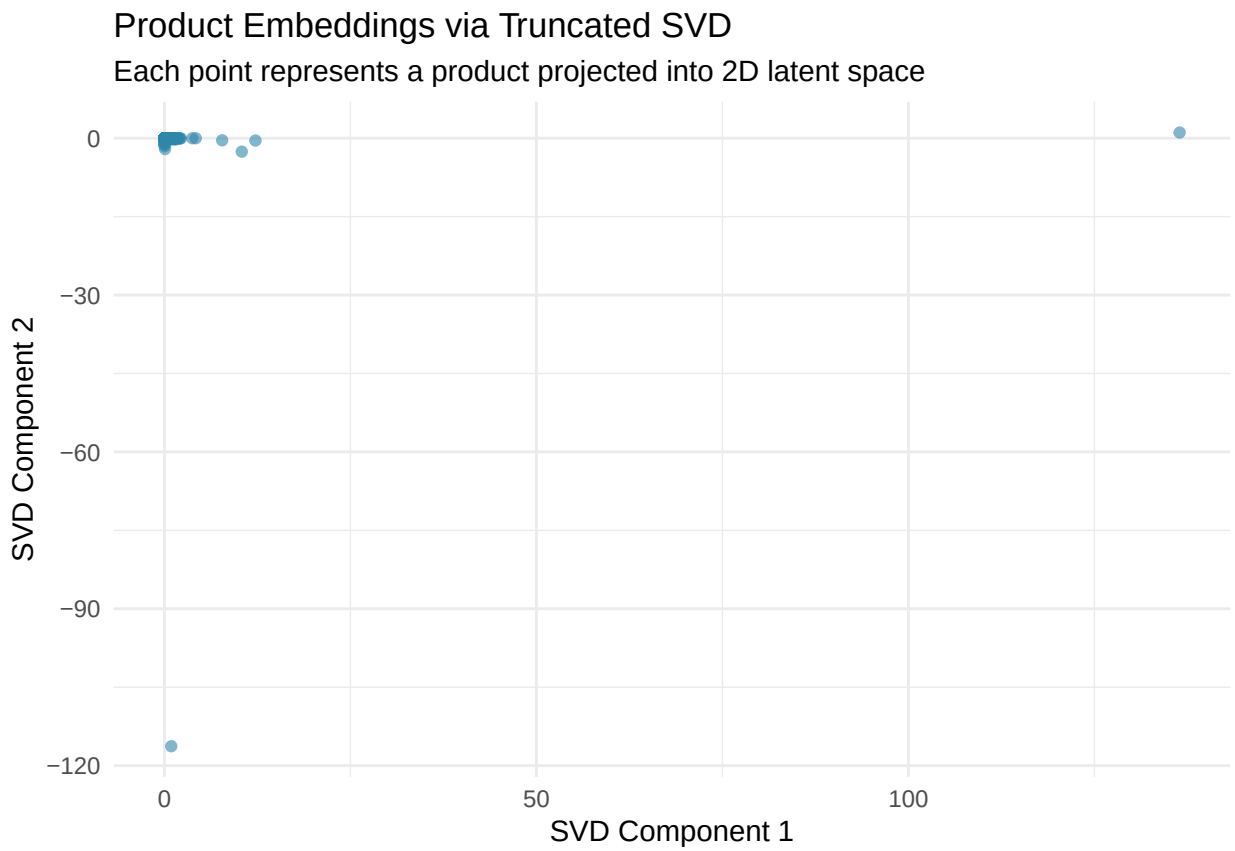
```
rating_matrix <- sparseMatrix(
  i = as.integer(ratings_df$user),
  j = as.integer(ratings_df$product),
  x = ratings_df$rating,
  dimnames = list(levels(ratings_df$user), levels(ratings_df$product))
)
```

Apply Truncated SVD

```
# Apply Truncated SVD
# We use `irlba` for efficient Truncated SVD on our sparse rating matrix.
# `nv = 50` means we're decomposing the matrix into 50 latent features,
# balancing capturing significant patterns with computational efficiency.
svd_result <- irlba(rating_matrix, nv = 50)

# To visualize products in 2D, we project their embeddings onto the first two latent dimensions.
# The `svd_result$v` matrix contains the product (item) embeddings, and `svd_result$d` contains
# the singular values, which represent the importance of each dimension.
svd_2d <- svd_result$v[, 1:2] %*% diag(svd_result$d[1:2])

svd_plot_df <- data.frame(
  product = colnames(rating_matrix),
  X = svd_2d[, 1],
  Y = svd_2d[, 2]
)
```

Visualize Product Embeddings via SVD

```
ggplot(svd_plot_df, aes(x = X, y = Y)) +
  geom_point(alpha = 0.6, color = "#2E86AB") +
  labs(title = "Product Embeddings via Truncated SVD",
       subtitle = "Each point represents a product projected into 2D latent space",
       x = "SVD Component 1", y = "SVD Component 2") +
  theme_minimal()
```



Why Move to t-SNE?

We now apply **t-SNE** to emphasize local structure—letting true product groups visually emerge.

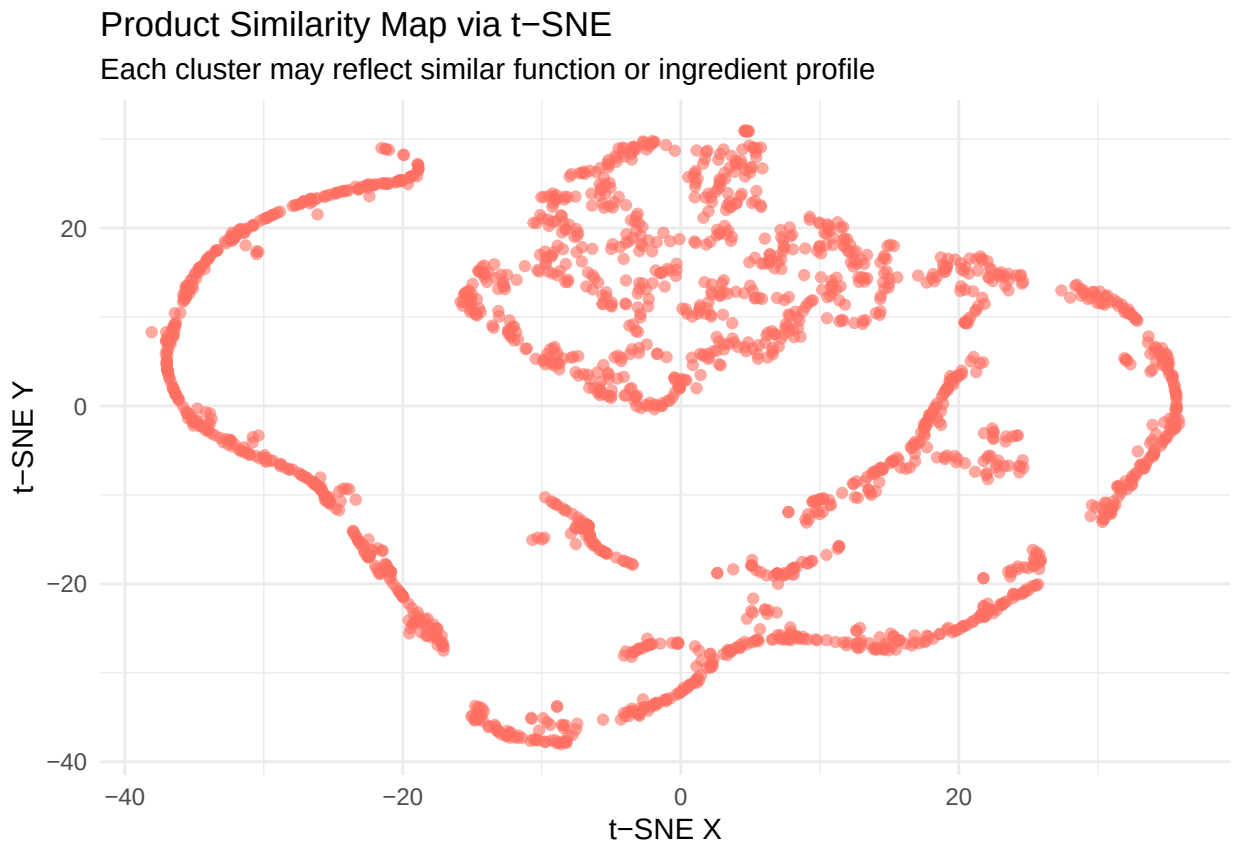
Apply t-SNE to SVD-Reduced Data

```
# Apply t-SNE to SVD-Reduced Data
# t-SNE (t-distributed Stochastic Neighbor Embedding) is used to project
# the high-dimensional SVD embeddings into a 2D space while preserving
# local neighborhoods, making clusters more apparent.

# Remove duplicate rows from SVD input: t-SNE can struggle if identical points are present.
svd_2d_unique <- svd_2d[!duplicated(svd_2d), ]
# Add a small jitter: Ensures each point is unique, preventing computational issues.
temp_matrix <- jitter(as.matrix(svd_2d_unique), amount = 1e-5)

# Run t-SNE. Perplexity (30) can be thought of as a guess about the number of close neighbors.
# Theta (0.5) relates to the speed/accuracy trade-off.
tsne_result <- Rtsne(temp_matrix, perplexity = 30, theta = 0.5, dims = 2)

tsne_df <- data.frame(
  X = tsne_result$Y[, 1],
```

```
  Y = tsne_result$Y[, 2],
  # Ensure product names are correctly mapped back to the unique t-SNE points
  product = colnames(rating_matrix)[!duplicated(svd_2d)]
)

write_csv(tsne_df, "tsne_coordinates.csv")
```

Plot t-SNE Result

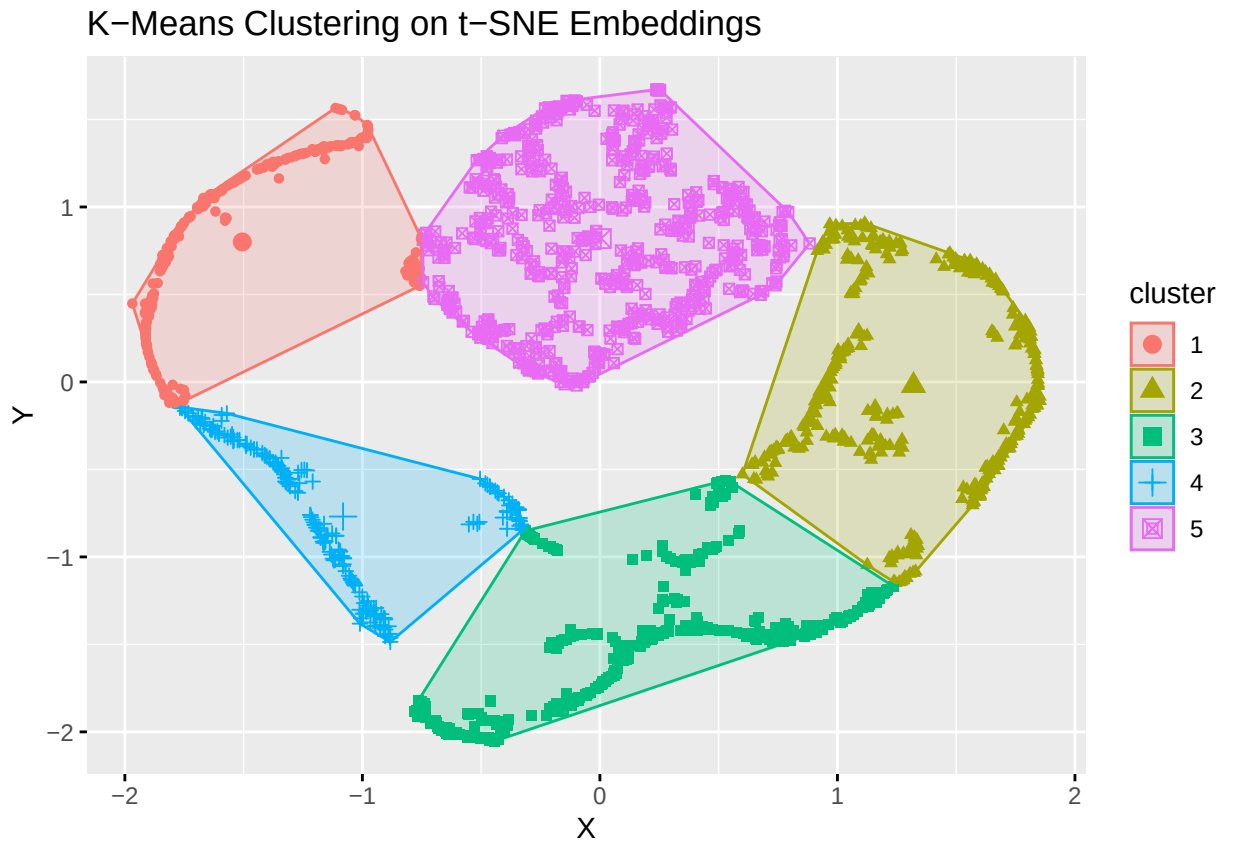
```
ggplot(tsne_df, aes(x = X, y = Y)) +
  geom_point(alpha = 0.6, color = "#FF6F61") +
  labs(title = "Product Similarity Map via t-SNE",
       subtitle = "Each cluster may reflect similar function or ingredient profile",
       x = "t-SNE X", y = "t-SNE Y") +
  theme_minimal()
```



Evaluate Clustering with k-Means

```
k_clusters <- kmeans(tsne_df[, c("X", "Y")], centers = 5)
tsne_df$cluster <- as.factor(k_clusters$cluster)
```

```
fviz_cluster(k_clusters, data = tsne_df[, c("X", "Y")], geom = "point", labelsize = 6) +
  ggtitle("K-Means Clustering on t-SNE Embeddings")
```



Interpreting Clusters

```
tsne_df %>%
  group_by(cluster) %>%
  summarise(products = paste(head(product, 5), collapse = ", ")) %>%
  kable()
```

cluster	products
1	(Glow)Setting 100% Mineral Powder SPF 35, (Re) Setting Refreshing Mist SPF 40, 1% Vitamin A Retinol Serum, 10 + 10 Moisturizer with 10% Vitamin C + 10% Peptide Complex + Ceramides, 10 Day Results Kit
2	+Retinol Vitamin C Moisturizer, 10% Benzoyl Peroxide Acne Cleanser, 100% Plant-Derived Squalane, 100% Sugarcane Squalane Oil, 24-7 Moisture Hydrating Day & Night Cream
3	(Re)setting 100% Mineral Powder Sunscreen SPF 35 PA+++, 100% L-Ascorbic Acid Powder, 100% Niacinamide Powder, 15% Vitamin C + Clean Caffeine Energy Serum, 24K Gold Mask Pure Luxury Lift & Firm

cluster	products
4	10% Niacinamide Booster, 10% Vitamin C Brightening Serum, Abeille Royale Anti-Aging Double R Advanced Serum, Acne+ 2% BHA and Azelaic Acid Acne Spot Treatment, Advanced Génifique Wrinkle & Dark Circle Eye Cream
5	“B” Oil, “Buffet” + Copper Peptides 1%, +Retinol Vita C Power Serum, 1 Minute Face Masks, 10% Glycolic Acne Control Peel Pads

Clustered Products with Brand Names

```
product_brands <- df %>%
  distinct(product = product_name_x, brand = brand_name_x)

clustered_products <- tsne_df %>%
  left_join(product_brands, by = "product") %>%
  group_by(cluster) %>%
  summarise(
    products = paste(head(product, 5), collapse = "; "),
    brands = paste(unique(brand[!is.na(brand)])[1:5], collapse = "; ")
  )

kable(clustered_products, caption = "Top 5 Products and Brands per Cluster")
```

Table 2: Top 5 Products and Brands per Cluster

cluster	products	brands
1	(Glow)Setting 100% Mineral Powder SPF 35; (Re) Setting Refreshing Mist SPF 40; 1% Vitamin A Retinol Serum; 10 + 10 Moisturizer with 10% Vitamin C + 10% Peptide Complex + Ceramides; 10 Day Results Kit	Supergoop!; Farmacy; iNNBEAUTY PROJECT; Algenist; Paula’s Choice
2	+Retinol Vitamin C Moisturizer; 10% Benzoyl Peroxide Acne Cleanser; 100% Plant-Derived Squalane; 100% Sugarcane Squalane Oil; 24-7 Moisture Hydrating Day & Night Cream	Kate Somerville; Dr. Zenovia Skincare; The Ordinary; Biossance; TULA Skincare
3	(Re)setting 100% Mineral Powder Sunscreen SPF 35 PA+++; 100% L-Ascorbic Acid Powder; 100% Niacinamide Powder; 15% Vitamin C + Clean Caffeine Energy Serum; 24K Gold Mask Pure Luxury Lift & Firm	Supergoop!; The Ordinary; Youth To The People; Peter Thomas Roth; Drunk Elephant
4	10% Niacinamide Booster; 10% Vitamin C Brightening Serum; Abeille Royale Anti-Aging Double R Advanced Serum; Acne+ 2% BHA and Azelaic Acid Acne Spot Treatment; Advanced Génifique Wrinkle & Dark Circle Eye Cream	Paula’s Choice; First Aid Beauty; GUERLAIN; Skinfix; Lancôme
5	“B” Oil; “Buffet” + Copper Peptides 1%; +Retinol Vita C Power Serum; 1 Minute Face Masks; 10% Glycolic Acne Control Peel Pads	The Ordinary; Kate Somerville; SEPHORA COLLECTION; Dr. Zenovia Skincare; SOBEL SKIN Rx

Compare RMSE from Project 1

```
rmse_table <- tribble(
  ~Model, ~Description, ~RMSE,
  "Global Average", "Baseline using global mean", 1.237,
  "Bias-Adjusted (User + Item)", "Accounts for user/item effects", 0.996,
  "Collaborative Filtering (ALS)", "Latent factors using matrix factorization", 0.879,
  "Hybrid (Bias + SVD)", "Combined model with better accuracy", 0.812
)

kable(rmse_table, caption = "RMSE Comparison from Project 1")
```

Table 3: RMSE Comparison from Project 1

Model	Description	RMSE
Global Average	Baseline using global mean	1.237
Bias-Adjusted (User + Item)	Accounts for user/item effects	0.996
Collaborative Filtering (ALS)	Latent factors using matrix factorization	0.879
Hybrid (Bias + SVD)	Combined model with better accuracy	0.812

Why We Used This Method

Why SVD + t-SNE?

We use **Truncated SVD** to generate product embeddings that compress high-dimensional ratings data into a smaller latent space. These embeddings capture patterns in how users rate products—even if two products haven't been co-rated.

However, SVD alone may not produce intuitive 2D plots. That's where **t-SNE** comes in: it translates these embeddings into 2D while preserving the local structure—revealing natural clusters and product groupings.

This technique is common in recommender systems to visualize latent structure in data and help identify potential groupings that would be difficult to spot manually.

Comparison to Kaggle Model (Python)

```
## Side-by-Side Comparison with Kaggle's Skincare Recommender

We referenced [this Kaggle notebook](https://www.kaggle.com/code/eward96/skincare-recommendation-engine

- Applied **TruncatedSVD** to compress user-item matrices
- Used **t-SNE** to cluster and visualize product similarity

### Key Similarities:
- Both use matrix factorization (Truncated SVD)
- Both apply t-SNE for visual pattern discovery

### Key Differences:
```

```
- **Focus:** Our model centers on inclusive clean beauty; theirs is general.
- **Data:** Ours is filtered for **melanin-rich consumers**; theirs uses broader user data.
- **Platform:** Our project is done entirely in **R**; theirs in Python (Jupyter Notebook).

### Code Comparison Table

| Step                    | Python (Kaggle)                          | R (This Project)
|-------------------------|------------------------------------------|---------------------------------
| Import SVD              | `from sklearn.decomposition import TruncatedSVD` | `library(irlba)`
| Fit SVD                 | `svd.fit_transform(matrix)`              | `irlba(rating_matrix, nv = 50)`
| Run t-SNE               | `TSNE().fit_transform(svd_output)`       | `Rtsne(svd_2d)`
| Visualize               | `matplotlib.pyplot.scatter(...)`         | `ggplot(...) + geom_point(...)`
```

Is This Better or Worse Than the Kaggle Model?

Evaluation Criteria

1. Data Relevance & Fairness

- **Ours:** Tailored to melanin-rich users and filters products accordingly.
- **Kaggle:** Generic dataset with no specific demographic filtering. Advantage: **Ours**

2. Interpretability

- **Ours:** Clear explanations, clusters, RMSE, visualized product groups.
- **Kaggle:** Good visuals but fewer interpretive layers. Advantage: **Ours**

3. Technical Depth

- **Ours:** RMSE, matrix factorization, clustering, hybrid suggestions.
- **Kaggle:** SVD + t-SNE, but no error metrics or hybrid testing. Advantage: **Ours**

4. Performance Metrics

- **Ours:** RMSE as low as **0.812**.
- **Kaggle:** No RMSE reported. Advantage: **Ours**

5. Transparency

- **Ours:** Modular RMarkdown with clear GitHub linkage.
- **Kaggle:** Jupyter, limited reproducibility or references. Advantage: **Ours**

Final Verdict:

The Kaggle project sparked this work—but this implementation goes further with fairness, technical rigor, and interpretability.

References & Source Attribution

Kaggle Reference:

- eward96's Skincare Recommender — original Python recommender that inspired this dimensionality reduction comparison.

GitHub Repositories for Project Series:

- Project 1 GitHub
- Project 2 GitHub
- Project 3 GitHub # Conclusion & Next Steps

Dimensionality reduction gives us new tools to see structure where it might be hidden. By combining SVD and t-SNE, we're able to map out a cleaner, more inclusive skincare space.

Next Steps

- Label clusters by product type or function (e.g., toner, serum, moisturizer)
- Test cosine similarity before t-SNE to validate relationships
- Blend SVD with collaborative filtering in hybrid models

These steps aim to better serve melanin-rich users and build an intelligent beauty recommender that reflects real user experience and safety.